

Interleaving and Race Conditions

When we write sequential code, debugging is simple. If our program crashes on line 10, we know with **100%** certainty that lines 1 through 9 executed perfectly in order. But when we write concurrent code, we lose this certainty. If we have two Goroutines running simultaneously, their instructions are **interleaved**.

1. What is Interleaving?

Interleaving refers to the order in which the CPU executes instructions from multiple concurrent tasks.

- Because the OS and Go Scheduler are constantly pausing one Goroutine to let another run, the exact order of execution is unpredictable.
- **Example:** If Task 1 has instructions [A, B] and Task 2 has instructions [X, Y], the computer might execute them as A, X, B, Y. The next time you run the same program, it might execute as A, B, X, Y.

2. The Machine Code Problem

It is more chaotic than it looks because it happens at the Machine Code level, not the Go code level.

- As **a = b + c**, requires 4 separate CPU instructions (Load B, Load C, Add them, Store in A).
- The Go Scheduler might execute "**Load B**" and "**Load C**", but then pause the task right in the middle of the equation to let another Goroutine run!

3. Race Conditions (The Ultimate Bug)

Race Condition occurs when the outcome of a program changes depending on the unpredictable interleaving order. It happens when two concurrent tasks attempt to **communicate** by reading and writing to the same shared variable.

The Example: Imagine Task 1 reads variable x, adds 1 to it, and writes it back. Task 2 prints variable x.

- If Task 1 finishes before Task 2 starts, the program prints 2.
- If the Scheduler decides to run Task 2 first, the program prints 1.
- The code is now fundamentally broken because it is unpredictable.

4. The Balance of Communication

Web Servers: A server spins up a new concurrent thread for every single user. But if two users look at the same product page and buy the last item in stock at the same millisecond, those two threads must communicate to update the inventory counter safely.

Expert Takeaway

Race conditions are the reason why companies lose millions of dollars. Imagine a banking app where two concurrent requests try to withdraw \$100 from an account with only \$150 in it. If the server interleaves the instructions poorly, both requests might read the balance as \$150 before either has a chance to deduct the money, resulting in the bank giving away \$200!